Agricultural Commodity Price Forecasting and Market Analysis

Using Hadoop HDFS, Apache Spark, MLlib, and Apache Kafka

Figures: docs/VERIFIED_METRICS.md (corrected 2026-10-23)

Final Review — Team AG-DATA

Name	Register no.
C Rishitha	CB.AI.U4AID24109
Karthikeya Y	CB.AI.U4AID24163
Ashwin Tyagi	CB.AI.U4AID24167
Kaushal S	CB.AI.U4AID24168

23AID302 — Big Data Analytics

Department of Artificial Intelligence and Data Science · Amrita Vishwa Vidyapeetham

Faculty Guide: Dr. Sreeja

1. What Are We Trying to Find?

Mandi: what price will be reported?

Forecast the modal price a mandi will report 1, 3, 7 and 14 days ahead: 36 commodities, 11,845 series.

Each forecast has an 80% conformal interval with measured coverage.

Baseline: persistence ("last price"); 17–34% of price changes are exactly zero.

Decision Support: outlook cards

Forecasts and intervals become cautious cards:

- **SELL:** even the upper interval estimate is not above the last price.
- **HOLD:** even the lower estimate exceeds the last price plus storage costs.
- **WATCH:** no clear signal (the default).
- **COMPARE:** a nearby mandi recently reported a clearly higher price.

Why do we need big data tools?

75.98 million archived reports and 21.06 million per-series window-feature rows do not fit single-node Pandas on a 16 GB laptop. HDFS stores the layers in replicated blocks; Spark splits the work across executor threads.

Goal: prepare the data, compare forecasting models honestly, and stream daily reports.

2. Our Data and the 5 Vs

Data layers

Layer	Rows	Note
Raw archive	75,984,017	2001–2026
Archive silver	75,644,801	quality + dedup
Official back-fill	7,720,662	2025-10 → 2026-10
Silver v2	82,114,083	archive + official
Forecast panel	21.06 M	11,845 series

Attributes: State, District, Market, Commodity, Variety, Grade, Arrival_Date, Min/Max/Modal Price.

Coverage finding: the archive mirror holds only 30.0% of official reports for 2025-10 → 2026-04; where both exist, 99.33% of modal prices agree.

5 Vs in our project

5 Vs	In our project
Volume	75.98M raw archive rows; 82.1M silver v2 rows; 692 MB silver Parquet in HDFS.
Velocity	Daily reports through Kafka topic `agri.prices.v1` into Structured Streaming.
Variety	Archive CSV files and two official data.gov.in API resources.
Veracity	Quality rules, de-duplication, quarantine; 30% archive coverage found.
Value	Evaluated price forecasts with intervals and cautious decision cards.

All models are trained and tested on real data only. A v1 synthetic workload exists for scalability tests and is never used for any model.

3. System Setup and Configuration



HDFS storage settings

- 1 NameNode + 3 DataNodes (Docker, one host).
- Block size: 128 MiB; replication: 3 copies of each block.
- Silver: 692 MB, HEALTHY; readable with 1 DataNode down.
- Snappy Parquet, partitioned by `year=`.

Spark application settings

- Engine: Apache Spark 4.2 in `local[*]` mode (10 cores).
- Multi-core local CPU threads (NOT YARN worker executors).
- Shuffle partitions: 64 in the forecast jobs, coalesced by AQE.
- Also: Docker Spark cluster, 2 workers $\times$ 3 cores.

Are HDFS DataNodes also running as Spark executors?

No. HDFS (Docker) only stores blocks, replication 3; Spark runs in `local[*]` mode on host cores. The DataNodes share one disk: this shows HDFS behaviour, not hardware durability.

4. Spark Workflow and the DAG

Steps in Spark

1. Read and parse CSVs with explicit StructType schemas.
2. Quality rules and business-key de-duplication → silver.
3. Window features per series ($\leq t$) and targets ($> t$).
4. Fit MLlib pipelines, choose on 2024, score the test once.

Partitions: 64 shuffle partitions (AQE coalesces); `year=` directory pruning.

Scaling (measured): full 75.6M rows, 10 cores: W1 42.2 s, W2 65.9 s; speed-up 2.4× / 3.0×.

DAG: a map of the work Spark runs

Execution DAG (Shuffle-Delimited Stages):



Arrows show stage dependencies.

Wide transformations create shuffle boundaries.

Exact stage counts are read from the Spark UI for each run.

Where do the stages come from?

Narrow transformations pipeline within a stage. Wide transformations (`Window.partitionBy`, `groupBy`) need cross-partition shuffles, which create stage boundaries. Spark numbers stages at runtime.

5. Scala Code and Streaming

Scala DataFrame API: source excerpt

```
// 1. Non-fatal date parsing
expr("try_to_date(trim(Arrival_Date), 'yyyy-MM-dd
    ')")
// 2. Business-key de-duplication
val w = Window.partitionBy(
    DataQuality.businessKey.map(col): _*)
  .orderBy(/* deterministic quality order */)
df.withColumn("_rn", row_number().over(w))
// 3. Leakage-safe features (Features.scala)
def days(lo: Long, hi: Long) = byEntity.
     rangeBetween(lo, hi)
.withColumn("mean30", avg(lp).over(days(-29, 0)))
```

What the code does

- **Schema Enforcement:** explicit StructType schemas.
- **Resilient Parsing:** a malformed date becomes null, not a crash.
- **Deterministic Dedup:** one reproducible row per business key.
- **No Lookahead:** feature windows end at the current report; targets start after it; a perturbation test enforces this.
- **Shared Code:** batch and stream use one canonicalisation.

How do we use Structured Streaming?

Kafka topic `agri.prices.v1` → `PriceEventStream`: bronze (raw + offsets), quarantine before a 10-day watermark, de-dup by `event_id`, silver and alerts. Effectively-once output per offset range.

6. How We Train and Test the Models

Baselines and models

Baselines

- Persistence (last report), moving averages, seasonal naive.
- Target: log return to a report strictly after the origin.

Spark MLlib pipelines

- Linear regression, random forest, gradient-boosted trees.
- One global model per horizon (next, 3, 7, 14 days).

Selected: GBT, absolute loss (next, 3 d); random forest, 60 trees, depth 10 (7, 14 d).

Strict temporal split & tuning

Chronological split (purged on target dates):

- Train: origins 2016-01-01 .. 2023-12-31.
- Validation: 2024, model choice and conformal calibration.
- Test: 2025-01-01 .. 2026-04-20, scored once.

Then, frozen models (no refitting):

- Same test period on complete official data.
- Prospective 2026-04-22 $\rightarrow$ 2026-10-22: data that did not exist in the archive.

Why keep test data aside chronologically?

Random k-fold cross-validation lets future market information leak into training. Splitting chronologically and scoring the test once tests the models on genuinely later periods, as in deployment.

7. Results

Forecasting — test 2025-01-01 .. 2026-04-20 (rel. MAE vs persistence)

Horizon	Persistence MAE	Model MAE	Rel. MAE	80% coverage	DM p	Prospective
Next report	189.0	183.5	**0.971**	78.6%	1.3e-22	0.963
3 days	242.6	231.1	**0.953**	79.4%	3.6e-47	0.955
7 days	308.8	289.7	**0.938**	80.0%	4.4e-39	0.958
14 days	397.0	366.4	**0.923**	80.5%	8.7e-29	0.952

MAE in Rs/quintal. Prospective = frozen models, 2026-04-22 → 2026-10-22, relative MAE.

What the results mean

- **3–8% lower error than persistence, significant at every horizon.**
- Better than persistence on 58.5–76.4% of individual series.
- 80% intervals cover 78.6–80.5% of outcomes.
- Cards (7 d): WATCH 98.5% of days; HOLD 0.5% (hit 80.0%); SELL 1.0% (hit 67.8%).
- Anomaly flags, injected corruptions only: PR-AUC 0.704 vs 0.352 (legacy IQR). No real-anomaly accuracy is claimed.

Streaming and storage results

Replay: 21,524 Kafka messages.

- 20,000 to silver + 1,317 duplicates removed + 207 quarantined; 0 missing.
- Batch/stream parity: 0 mismatches.
- Producer→sink latency p50: 1.69 s.
- Live pull 2026-10-23: 4/4 records to silver.
- HDFS: 692 MB silver, replication 3, HEALTHY.

8. Challenges, Limits and Next Steps

Challenges we handled

- JDK 21 reflection blocked Spark; solved via JVM `-add-opens` flags.
- Malformed dates handled by `try_to_date` instead of failing jobs.
- Persistence is hard to beat (17–34% zero changes): absolute-loss GBT was needed.
- The archive mirror missed 70% of recent reports; we back-filled official data.

Limits and next steps

- Local Spark (`local[*]`) and a one-host Docker cluster, not multi-node YARN.
- HDFS replicas share one disk; single NameNode (no HA).
- Gains over persistence are modest; per-series smoothing matches us at 1–3 days.
- No public deployment yet; decision-card hit rates are not farm profits.

What changed since v1, and why?

v1 reported an anomaly classifier with very high accuracy and Kafka speed figures. No code or log produces them, and the v1 labels leaked, so they are withdrawn (`docs/AUDIT_PRIOR_RESULTS.md`). We now lead with an honestly evaluated forecasting benchmark.

Conclusion: 3–8% lower error than persistence at all horizons, calibrated 80% intervals, and gains that hold on six unseen months.

9. References

1. **Apache Spark: DataFrames, machine learning and Structured Streaming.**
   https://spark.apache.org/docs/4.2.0/
2. **Apache Hadoop: HDFS distributed storage architecture.**
   https://hadoop.apache.org/docs/r3.3.6/
3. **Agmarknet: Indian agricultural market daily price portals (2001–2026).**
   https://agmarknet.gov.in/
4. **Chambers, B., & Zaharia, M. (2018). Spark: The Definitive Guide. O'Reilly Media.**
   https://www.oreilly.com/library/view/spark-the-definitive/9781491912201/

Project evidence: GitHub repository https://github.com/kaushalrog/AG-DATA; every figure with its source file in docs/VERIFIED_METRICS.md.

Thank you